

macOS Active-Usage Tracker

Concept paper — per-application time measured only while the keyboard or mouse is actually being used.

1. Problem statement

Classic “time-in-app” trackers count every second an application is frontmost, even if the user walked away from the Mac. The goal of this system is to report **effective** time: for each program, how long the user was **actively producing input** (keystrokes or mouse activity) while that program was in focus. This gives a realistic picture of where attention goes and filters out idle “wall-clock” noise.

2. Core concepts

- **Foreground app** — the *NSRunningApplication* that owns the key window at a given moment.
- **Input event** — a keyDown / keyUp, mouseMoved, mouseDown, scrollWheel or trackpad gesture delivered by *CGEventTap*.
- **Active interval** — a contiguous stretch of time during which the same app was frontmost *and* at least one input event occurred within the idle-cutoff window (default 30 s).
- **Heartbeat** — a 1-second timer that folds raw events plus frontmost-app state into the aggregator and closes an interval as soon as the user goes idle or switches apps.

3. Architecture

The system is a single-machine, local-only pipeline. A lightweight Swift agent runs as a user LaunchAgent, captures OS-level signals, writes them to a SQLite database, and exposes them through a SwiftUI menu-bar app.

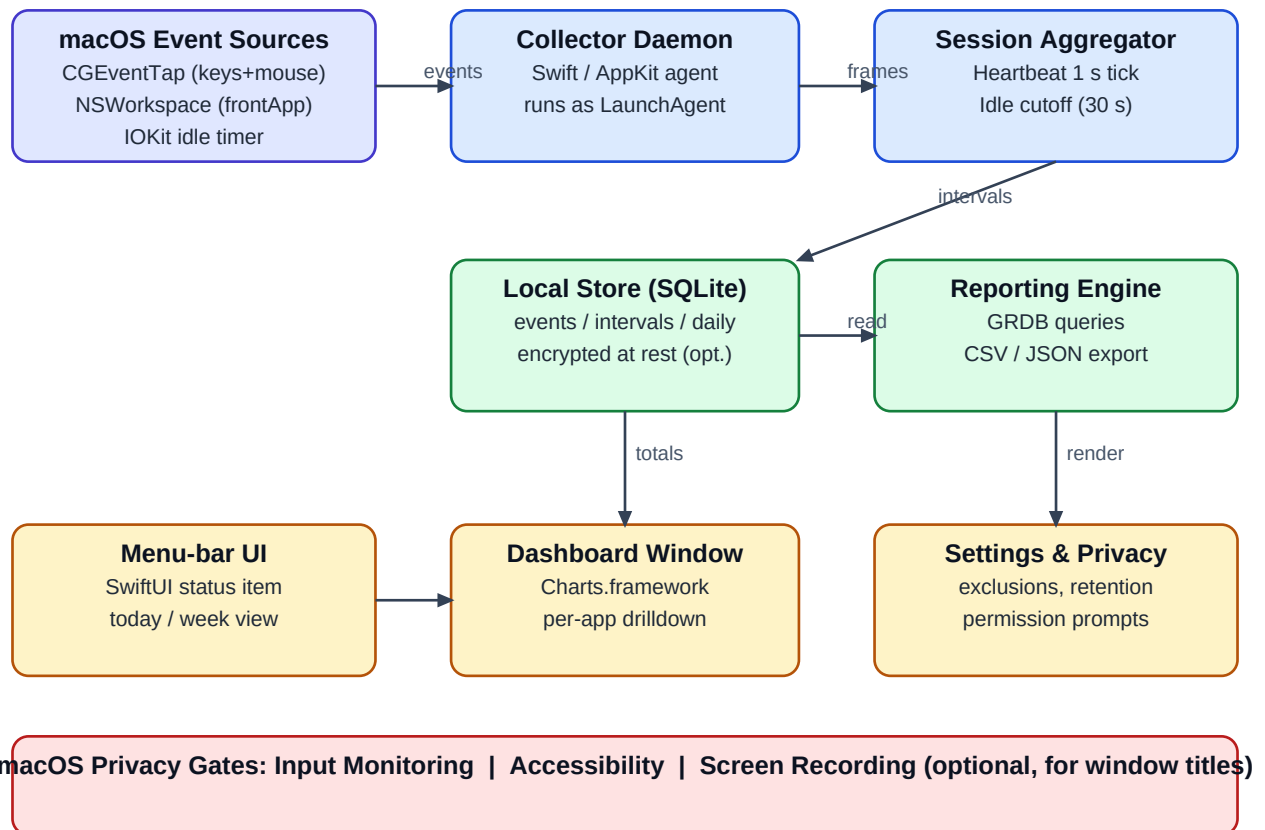


Figure 1 — high-level data flow. Red band at the bottom marks the macOS privacy prompts the user must grant.

3.1 Pipeline stages

- **Event sources.** *CGEventTap* at *KCGHIDEEventTap* delivers global keyboard and mouse events; *NSWorkspace* notifications publish frontmost-app changes; *IOKit* *HIDIdleTime* gives a second opinion on idleness.
- **Collector daemon.** A background actor that fan-ins the three sources, never looks at event content (only counts / timestamps), and forwards minimal records to the aggregator.
- **Session aggregator.** Maintains the open *active interval* for the current app, ticks once per second, closes the interval on app-switch or idle-timeout, and flushes it to the store.
- **Local store.** SQLite via GRDB. Three tables (events, active_intervals, daily_totals) with daily rollups computed by a nightly task to keep queries cheap.
- **Reporting engine.** Pure-Swift query layer that feeds both the UI and an optional CSV/JSON exporter.
- **UI.** Menu-bar status item with a “today” glance plus a full dashboard window with daily / weekly / per-app breakdowns.

4. Data model

Table	Columns	Purpose
events	ts, bundle_id, kind (key mouse), count	Raw per-second counters. Short retention (e.g. 7 days).
active_intervals	start_ts, end_ts, bundle_id, app_name, key_count, mouse_count	Time intervals of active time; active time & "fact" table.
daily_totals	date, bundle_id, active_seconds, keys, mouse_events	Pre-aggregated for fast charts.
apps	bundle_id, display_name, category, excluded	User-curated metadata and opt-outs.

5. Heartbeat algorithm

```
every 1 s tick:
    now = monotonic_clock()
    frontApp = NSWorkspace.frontmostApplication
    lastInput = max(ts of any key / mouse event seen so far)
    idleFor = now - lastInput

    if idleFor > IDLE_CUTOFF or frontApp != openInterval.app:
        close(openInterval, end_ts = min(now, lastInput))
        openInterval = nil

    if idleFor <= IDLE_CUTOFF:
        if openInterval == nil:
            openInterval = new Interval(app=frontApp, start=lastInput)
        openInterval.keys += keysThisTick
        openInterval.mouse += mouseThisTick

flush(openInterval) every 30 s as a crash-safety checkpoint
```

The algorithm guarantees that time is only attributed to an app while input is actually happening, and that a single walk-away never inflates the totals by more than IDLE_CUTOFF seconds.

6. Software stack

Layer	Choice	Why
Language	Swift 5.10	First-class AppKit / SwiftUI access, tight binary, no runtime.
Event capture	CGEventTap + NSEvent monitors	System-wide HID events without polling; counts only, no payload.
Frontmost app	NSWorkspace notifications	Push-based, no CPU cost between switches.
Idle detection	IOHIDManager HIDIdleTime	Screensaver-aware and matches what the OS reports.
Persistence	SQLite + GRDB.swift	Embedded, zero-config, great Swift API, WAL-safe.
UI	SwiftUI + Swift Charts	Menu-bar app, native look, built-in charting.
Packaging	Signed .app + LaunchAgent plist	Auto-start at login, notarised, sandbox-compatible.
Permissions	Input Monitoring + Accessibility	Required by CGEventTap; requested on first run via TCC.
Build / CI	Xcode 15, Swift Package Manager, GitHub Actions	Standard Apple toolchain; signed builds on tag.

7. Privacy & security

- **No keylogging.** The collector stores *counts* of events per second, never key codes or characters.
- **Local only.** The database lives under ~/Library/Application Support/MacUsageRep; nothing leaves the machine unless the user explicitly exports.
- **Exclusions.** Users can blacklist bundle IDs (e.g. password managers, banking apps) — excluded apps produce no rows.
- **Retention.** Raw events are purged after 7 days; active_intervals after 180 days; daily_totals kept indefinitely (they are already aggregated).
- **At-rest encryption.** Optional SQLCipher build for users who want the DB encrypted with a Keychain-held key.
- **Transparency.** Menu-bar icon always visible; a single click opens the dashboard so the user can see what is recorded right now.

8. Reporting & UX

- **Today glance.** Top-5 apps by active seconds since midnight, live-updating.
- **Daily view.** 24-hour timeline coloured by app, with gaps for idle periods.
- **Weekly & monthly.** Stacked bars per day, per-app totals, trend lines.
- **Drilldown.** For a selected app: keys vs mouse ratio, most active hours.
- **Export.** CSV / JSON / Markdown summary for any date range.

9. Roadmap & extensions

- **Window-level granularity.** Capture the frontmost window title (requires Screen Recording permission) to split e.g. “Chrome / docs” vs “Chrome / YouTube”.
- **Category rules.** User-defined tags (work, comms, leisure) with automatic rollups.
- **Focus insights.** Detect long uninterrupted intervals as “flow sessions” and surface them.
- **Multi-device sync.** End-to-end encrypted iCloud sync for users on several Macs.
- **Automations.** Shortcuts / CLI so other tools (e.g. invoicing, time-tracking) can pull today’s totals.

10. Open questions

- What is the right default IDLE_CUTOFF? 30 s is forgiving; 10 s is stricter but penalises reading.
- Should mouse movement alone (no clicks, no keys) count as active? Default: yes, but only when the delta exceeds a small threshold to ignore accidental bumps.
- How to treat full-screen media playback (user watching, not typing)? Candidate: separate “passive” bucket when the frontmost app declares NSActivityLatencyCritical and there is audio output.